

CORPSEED ERP — Application Documentation

Frontend Codebase Reference (folder “mainioclose”)

v1.0 — generated from source, branch latest-versionv7

Date: 12 August 2026

Prepared for: Avnish Kumar

Corpseed ERP — internal engineering documentation

Table of Contents

1. Introduction
2. Technology Stack
3. High-Level Architecture
4. Authentication & Session Flow
5. Role- and Department-Based Navigation
6. Routing Map
7. Complete Folder & File Structure
8. Module-by-Module Reference
9. State Management (Redux Toolkit)
10. End-to-End Business Flow (“Lead to Ledger”)
11. Build, Lint & Tooling

1. Introduction

This document describes the Corpseed ERP frontend application that lives in the “mainioclose” repository. It is a single-page React application that runs the day-to-day operations of the business — from the moment a sales lead is captured to the moment the related invoice is reconciled in the books of account.

The ERP is organised as one React app with many feature modules, each mapped to a top-level folder under `src/`. Every module talks to one of four backend microservices (`securityService`, `leadService`, `accountService`, `operationService`) over a single shared Axios client. Navigation, page access and the sidebar menu are all driven by the logged-in user's department and role, so different employees effectively see a different “slice” of the same application.

1.1 Purpose of this document

- Explain how the application starts up, authenticates a user, and routes them to the correct workspace.
- Map every business module (Sales, Accounts, Procurement, Operation, Quality, HR, Legal, Admin, Settings, Users, Vendor Requests, Dashboards) to its folder, files and routes.
- Describe the end-to-end business flow — how a Lead becomes a Company, a Project, a Purchase Order, an Invoice, and finally a ledger entry.
- Provide the complete folder and file structure of the source tree as a reference.

1.2 Who this is for

New engineers joining the frontend team, QA engineers who need to know which screen maps to which route/file, and product/operations stakeholders who want a plain-language walkthrough of how the ERP mirrors the real business process.

2. Technology Stack

The application is a Vite-built React 18 single-page app. Key libraries:

Concern	Library / Tool
Build tool / dev server	Vite 6 + vite-tsconfig-paths
UI framework	React 18.3 (functional components + hooks)
Routing	react-router-dom v6 (nested routes, layout routes, Outlet)

Global state	Redux Toolkit (@reduxjs/toolkit) + redux-persist
Component / cache keep-alive	react-activation (KeepAlive/AliveScope — keeps Leads list mounted across navigation)
UI component kit	HeroUI (@heroui/react) + Ant Design (antd) + Tailwind CSS 4
Forms & validation	react-hook-form + @hookform/resolvers + zod
Rich text editing	TipTap (@tiptap/*) and CKEditor 5 (proposal/template authoring)
Charts	Recharts (dashboards)
HTTP client	Axios (single instance in httpRequest.js)
Realtime	@stomp/stompjs + sockjs-client (WebSocket notifications)
PDF / document export	@react-pdf/renderer, jsPDF, html2canvas / html2canvas-pro, html2pdf.js
Spreadsheet export	xlsx, react-csv
Dates	dayjs, @internationalized/date
Language	JavaScript (.jsx) with a light TypeScript layer (tsconfig, .ts files, vite build runs `tsc`)

3. High-Level Architecture

3.1 Application bootstrap

The startup sequence, file by file:

1. main.jsx — mounts the React tree: Redux <Provider store> → <PersistGate> (redux-persist) → <UIProvider> (HeroUI theme provider, src/uiprovider.jsx) → <BrowserRouter> → <App />.
2. App.jsx — wraps everything in <AliveScope> (react-activation, enables KeepAlive) and a single top-level <Suspense fallback={<LoadingSpinner />}> boundary, then declares <Routes>. On mount it dispatches restoreSession() to rehydrate auth state from sessionStorage.
3. Public routes (no login required): / (HomePage), /login, /forgotPassword, /:email/otp, and /:leadId/:uuid/estimate-preview (a shareable, unauthenticated estimate preview link sent to prospective clients).
4. Everything else lives under /erp/:userId, guarded by <ProtectedRoute /> and rendered inside the shared <Layoutpage /> shell (sidebar + header + breadcrumbs + Outlet).
5. Inside that shell, every module contributes its routes via a dedicated *ModuleRouting.jsx file from src/routings/, all combined inside App.jsx.

3.2 Backend integration

All network calls go through one Axios instance created in `src/httpRequest.js`, with a relative baseURL ("/") and a Bearer token taken from `localStorage/sessionStorage`. The path prefix of each endpoint identifies which backend microservice handles it:

Path prefix	Service	Responsibility
/securityService/api/...	Security Service	Authentication (sign-in / OTP / password reset), user & role CRUD, department & designation master data.
/leadService/api/...	Lead Service	The largest service — leads, companies, estimates, proposals, quality/IVR, industry master data, products/solutions, most Settings screens, dashboards.
/accountService/api/...	Account Service	Accounts & finance: ledgers, vouchers, invoices, GST/TDS, credit/debit notes, organization books.
/operationService/api/...	Operation Service	Projects, vendor requests/quotations, purchase orders, procurement payment requests, operation settings.

3.3 Realtime notifications

`src/useNotificationSocket.js` opens a STOMP client over `WebSocket/SockJS` per logged-in user. `src/components/NotificationSocketProvider.jsx` wires it into context and `src/components/NotificationBell.jsx` renders the bell icon with unread-count badge shown in the top header of Layoutpage.

4. Authentication & Session Flow

Login and session handling is centralised in `src/toolkit/slices/authSlice.js` and `src/ProtectedRoute.jsx`.

1. User submits email/password on `src/login/Login.jsx`, which dispatches `getCurrentUser()` → POST `/securityService/api/auth/signin`.
2. On success, the response (`id, jwt, roles[], ...`) is written to `sessionStorage` as "userDetail" and into the Redux auth slice (`isAuth=true`).
3. `Login.jsx` immediately dispatches `getDepartmentOfUser(userId)` → GET `/leadService/api/v1/users/getSingleUserById` to fetch the user's department, and toggles the "auto-present" presence flag for the automation engine.
4. Redirect after login is role/department aware: ADMIN → `/erp/:id/dashboard`; department "sales" → `/erp/:id/sales/dashboard`; "procurement" → `/erp/:id/procurement/vendors-requests`; other departments fall through to their own dashboard/landing route (see §5).

5. On every subsequent app load, App.jsx dispatches `restoreSession()`, which reads `sessionStorage("userDetail")` back into Redux — this is what keeps a user logged in across a page refresh without hitting the server again.
6. `src/ProtectedRoute.jsx` guards the whole `/erp/:userId` subtree: it compares the `:userId` route param against `auth.currentUser.id`. Any mismatch (not logged in, wrong user id in the URL, stale/foreign session) triggers `logoutFun()` and a redirect to `/login`.
7. Forgot-password flow: `ForgotPassword.jsx` → `forgotPasswordApi()` → OTP screen (`Otp.jsx` at `/:email/otp`) → `updatePassword()`.
8. Logout clears `sessionStorage("userDetail", "vendorDetail")` and the `redux-persist "persist:root"` key (`src/toolkit/store.js` special-cases the `"auth/logoutFun"` action to wipe persisted state).

Note: *Redux persistence: `redux-persist` stores most slices in `localStorage` under `"persist:root"` (auth is explicitly blacklisted there), while the `auth` slice itself is persisted separately to `sessionStorage` under key `"auth"` — so a closed browser tab naturally logs the user out, while a refresh within the same tab does not.*

5. Role- and Department-Based Navigation

`src/layouts/Layoutpage.jsx` picks which sidebar menu (from `src/layouts/NavItems.jsx`) to render using `getNavItemsByDepartment(department, isAdmin)`. Any user with the ADMIN role always gets the full menu (all modules); everyone else gets only their department's slice.

User's department (lower-cased)	Nav item set used	Landing area
ADMIN role (any department)	<code>navItems</code> (full menu, every module)	<code>/dashboard</code>
sales	<code>salesNavItems</code>	<code>/sales/dashboard</code>
quality team	<code>qualityNavItems</code>	<code>/quality/dashboard</code>
accounts	<code>accountNavItems</code>	<code>/accounts/dashboard</code>
procurement	<code>procurementItems</code>	<code>/procurement/vendors-requests</code>
human resource	<code>hrItems</code>	<code>hr/usersList</code>
crt, technical, liaisoning	<code>operationNavItems</code>	<code>operation/projects</code>
legal	<code>legalNavItems</code>	<code>legal/operationsRequests</code>
operations, "liaisoning test", "crt test"	<code>operationEmplItems</code>	<code>/operation/dashboard</code>

Note: *The Legal menu (and every route beneath it) is additionally protected at the item level via `allowedRoles: ["ADMIN", "Legal Executive"]` inside `NavItems.jsx`.*

6. Routing Map

Every feature module owns one file in `src/routings/` that returns a fragment of `<Route>` elements; `App.jsx` composes all of them under the single `/erp/:userId` protected branch. The tables below list the concrete URL path, the component it renders and the source file, module by module.

6.1 Dashboards — `src/routings/DashboardRouting.jsx`

Path	Component
dashboard	AdminDashboards
dashboard/:monthDate/leadData	LeadDashboardDetail
sales/dashboard	SalesDashboard
sales/dashboard/:monthDate/leadData	LeadDashboardDetail
accounts/dashboard	AccountsDashboard
quality/dashboard	AdminDashboards (reused)
operation/dashboard	OperationsDashboard

6.2 Sales — `src/routings/SalesModuleRouting.jsx`

Top-level sales pages:

Path	Component
sales/allTask	AllTasks (kept alive)
sales/leads	Leads (kept alive — busiest screen in the app)
sales/leads/:parentLeadId/childLeads	ChildLead
sales/company	Company
sales/company/:companyId/gstDetails	CompanyGstList
.../gstDetails/:unitId/projects	CompanyProjects
.../gstDetails/leads	CompanyLeads
.../gstDetails/:stateName/companyUnits	CompanyUnits
.../companyUnits/:companyUnitId	CompanyUnitDetails (tabs: unitDetails, companyProjects, companyLeads)
sales/leadForm	CompanyForm

sales/estimate	Estimate
sales/allInvoice	AllInvoice
sales/proposal	AllProposal
sales/unbilled	SalesUnbill
sales/unbilled/:unbilledId/invoices	InvoicesByUnbilled
sales/discountedEstimate	DiscountedEstimate
sales/autoHistory	AutoHistory
sales/salesReport	SalesReport
sales/automationReport	AutomationStatus
sales/advanceTaxInvoice	SalesAdvanceInvoice
sales/poPayments	SalesPOPayment
sales/projects	SalesProject
sales/servingCompanies	ServingCompanies

The Lead Detail page (LeadDetail.jsx) is mounted as a nested layout route under every lead-list context — sales/allTask/:leadId, sales/leads/:leadId, sales/leads/:parentLeadId/childLeads/:leadId, the company→GST→unit→leads drill-down, and sales/autoHistory/:leadId. In every case it exposes the same set of nested tabs:

Tab path (relative)	Component	Purpose
leadDetail (index)	LeadInfo	Core lead info form
childLead	ChildLead	Split / linked child leads
companyForm	CreateCompanyForm	Convert lead → new company
leadCompanyForm	CreateLeadCompanyForm	Attach lead to an existing company
vendors	Vendors	Vendors associated with this lead
proposal	Proposal	Draft / send proposal
leadEstimate	LeadEstimate	Estimate for this lead
leadEstimates	LeadEstimates	List of estimate versions
leadTasks	LeadTask	Follow-up tasks
leadHistory	LeadHistory	Full activity/audit timeline
paymentTerm	ServicePaymentTerm	Payment-term configuration

6.3 Quality — src/routings/QualityRouting.jsx

Path	Component
quality/leads	Leads (shared with Sales, kept alive)
quality/leadsSearch	LeadSearch
quality/leadsSearch/:leadId	LeadDetail (tabs: leadDetail, vendors, proposal, leadHistory)
quality/leads/:leadId	LeadDetail (tabs: leadDetail, companyForm, vendors, proposal, leadEstimate, leadHistory)
quality/ivr	IVR
quality/report	IVRReport
quality/leads/:leadId/leadHistory	LeadHistory

6.4 Accounts — src/routings/AccountsModuleRouting.jsx

Top-level accounts pages:

Path	Component
accounts/companyApprovals	CompanyApprovals
accounts/companyApprovals/:companyId/units	CompanyUnitsInAccount
accounts/expensesApprovals	Expense
accounts/vendorDetails	VendorDetails
accounts/creditNote	CreditNote
accounts/debitNote	DebitNotes
accounts/paymentApprovals	PaymentApprovals
accounts/companyForm	CompanyForm
accounts/orgEstimate	OrganizationEstimate
accounts/bankStatement	BankStatement
accounts/paymentRegister	PaymentRegister
accounts/allInvoice	AllInvoice
accounts/purchaseInvoices	PurchaseInvoices
accounts/advanceInvoices	AdvanceInvoices

accounts/unbilled	Unbill
accounts/unbilled/:unbilledId/invoices	InvoicesByUnbilled
accounts/vendorRestrictions	VendorRestrictions
accounts/taxation	Taxation
accounts/procurementPaymentRequests	ProcurementPaymentRequest
accounts/poPayments	PurchaseOrder
accounts/vendorsPayment	VendorPayments
accounts/vendorsPayment/:paymentId/paymentHistory	VendorPaymentHistory

Nested under accounts/organizations (Organizations.jsx as layout route):

Path	Component
(index)	OrganizationDetail
group	Group
group/:groupId/groupLedger	GroupLedger
ledger	Ledger
ledger/:ledgerId/entries	LedgerEntriesPage
ledger/:ledgerId/ledgerDetail	LedgerDetail
voucher	Voucher
dayBook	DailyBook
profitLoss	ProfitLoss
cashflow	CashFlow
balanceSheet	BalanceSheet
trailBalance	TrailBalance
tds	TDS
gst	GST
salesReport	SalesReport
settings/groups	LedgerType
settings/voucherType	VoucherType
settings/statutory	Statutory

Note: `accountLoginModuleRouting()` (same file) mounts a flattened, non-admin-friendly copy of the Organization pages directly under `accounts/organizations/*` and `accounts/settings/*` for users whose department is exactly “accounts”.

6.5 Industry — `src/routings/IndustryModuleRouting.jsx`

Path	Component
<code>industry/industries</code>	<code>Industries</code>
<code>industry/subindustries</code>	<code>SubIndustries</code>
<code>industry/categories</code>	<code>Categories</code>
<code>industry/businessActivity</code>	<code>BusinessActivity</code>

6.6 HR — `src/routings/HRModuleRouting.jsx`

Path	Component
<code>hr/usersList</code>	<code>UsersList</code>
<code>hr/usersApprovalList</code>	<code>UserApprovals</code>
<code>hr/services</code>	<code>Services</code>
<code>hr/services/:serviceId/rating</code>	<code>Rating</code>

6.7 Operation — `src/routings/OperationModuleRouting.jsx`

Path	Component
<code>operation/projects</code>	<code>Projects</code>
<code>operation/projects/:projectId/projectDetail</code>	<code>ProjectDetails</code>
<code>.../projectDetail/purchaseOrder</code>	<code>ProjectPurchaseOrder</code>
<code>.../purchaseOrder/:poid/procurementPaymentRequest</code>	<code>ProjectPR</code>
<code>.../projectDetail/activities</code>	<code>ProjectActivities</code>
<code>operation/legalRequests</code>	<code>LegalRequests</code>
<code>operation/approvals</code>	<code>ManagerApprovals</code>
<code>operation/reopenCloseApprovals</code>	<code>ForceCloserAndReopen</code>
<code>operation/expenses</code>	<code>Expenses</code>

operation/settings (index)	redirects to userMap
operation/settings/userMap	UserMapWithProduct
operation/settings/milestones	Milestone
operation/settings/allDocuments	Documents
operation/settings/departments	Departments

6.8 Procurement — src/routings/ProcurementRouting.jsx

Path	Component
procurement/vendor-quotations	ProcurementSolutions
.../vendor-quotations/:solutionId/detail (overview)	SolutionOverview
.../detail/vendors-data	VendorsData
.../detail/rfq	RequestForQuotation
.../detail/rfq/:rfqId/vendors	RFQVendors
.../rfq/:rfqId/vendors/:vendorId/quotations	Quote
procurement/vendorList	Vendors
procurement/vendors-requests	VendorRequests
.../vendors-requests/:requestId/:leadId/requestDetail	VendorRequestDetail
procurement/vendors-payments	VendorPaymentForProcurement
.../vendors-payments/:paymentId/paymentHistory	VendorPaymentHistory
procurement/vendors-estimates	VendorEstimate
procurement/projects	Projects (shared with Operation)
procurement/projects/:projectId/projectDetail	ProjectDetails
.../projectDetail/activities	ProjectActivities

6.9 ERP Settings — src/routings/ERPSettingRouting.jsx

Path	Component
settings/operations	OperationsSettings
settings/status	LeadStatus

settings/solutions	Solutions
.../solutions/:solutionId/detail (solutionPrice)	SolutionPrice
.../detail/documents	ProductDocument
.../detail/milestones	ProductMilestones
.../detail/serviceDetails	ProductServiceDetails
.../solutions/:solutionId/businessArrangement	BusinessArrangement
.../businessArrangement/:id/productCategory	ProductCategory
.../productCategory/:categoryId/subCategory	ProductSubCategory
settings/comments	LeadComments
settings/ipAddress	IpAddress
settings/slug	Slug
settings/country	CountryData
settings/country/state/:countryName	StateData
settings/country/state/:stateName/city/:stateName	CityData
settings/applicantType	ApplicantTypes
settings/urls	Urls
settings/department	Department
settings/designation	Designation
settings/procurementCategory	ProcurementCategory
.../procurementCategory/:categoryId/subcategory	ProcurementSubCategory
settings/proposalTemplate	TemplatesAndEmailBody
settings/menu	ProposalMenu
.../menu/:menuId/category	ProposalCategory
.../category/:categoryId/subcategory	ProposalSubCategory
.../subcategory/:subcategoryId/service	ProposalService
settings/paymentTerms	PaymentTems

6.10 Directly in App.jsx (Users, Admin, Legal, public routes)

Path	Component
/	HomePage
/login	Login
/forgotPassword	ForgotPassword
/:email/otp	Otp
/:leadId/:uuid/estimate-preview	EstimatePreview (public share link)
users/usersList	Users
users/usersMailConfig	UserMailConfig
users/deactiveUsersList	DeactiveUserList
users/approvalList	UserManagerApproval
users/usersList/:currentUserId/userHistory	UserHistory
admin/vendorPaymentApproval	VendorPaymentApproval
admin/discountedEstimate	DiscountedEstimateApproval
admin/vendorRestrictions	AdminVendorRestrictionApproval
admin/poApproval	AdminPoApproval
legal/operationsRequests	OperationsLegal
legal/vendors	ProcurementVendors
legal/companyDocuments	CompanyDocuments
legal/paymentVerification	PaymentVerification
/unauthorized	inline “Unauthorized” message

7. Complete Folder & File Structure

The full source tree of the application (src/ and public/), exactly as it exists in the repository. Folders are listed before files at each level.

7.1 public/

```
└─ public/  
  └─ CORPSEED.webp
```

7.2 src/

```
└─ src/  
  └─ accounts/  
    └─ ledgers/  
      └─ Ledger.jsx  
      └─ LedgerEntriesPage.jsx  
      └─ LedgerMasterSection.jsx  
    └─ organization/  
      └─ settings/  
        └─ LedgerType.jsx  
        └─ Statutory.jsx  
        └─ VoucherType.jsx  
      └─ AllInvoice.jsx  
      └─ BalanceSheet.jsx  
      └─ BankStatement.jsx  
      └─ CashFlow.jsx  
      └─ DailyBook.jsx  
      └─ DebitNotes.jsx  
      └─ Group.jsx  
      └─ GroupLedger.jsx  
      └─ GST.jsx  
      └─ InvoicesByUnbilled.jsx  
      └─ Ledger.jsx  
      └─ LedgerDetail.jsx  
      └─ OrganizationDetail.jsx  
      └─ OrganizationEstimate.jsx  
      └─ Organizations.jsx  
      └─ PaymentRegister.jsx  
      └─ ProfitLoss.jsx  
      └─ PurchaseInvoices.jsx  
      └─ SalesReport.jsx  
      └─ Taxation.jsx  
      └─ TDS.jsx  
      └─ TrailBalance.jsx  
      └─ Unbill.jsx  
      └─ Voucher.jsx  
    └─ vendorDetails/  
      └─ VendorDetails.jsx
```


- |— AdvanceInvoices.jsx
- |— AdvanceTaxInvoiceView.jsx
- |— CompanyApprovals.jsx
- |— CompanyForm.jsx
- |— CompanyUnitsInAccount.jsx
- |— CreditNote.jsx
- |— Expense.jsx
- |— PaymentApprovals.jsx
- |— ProcurementPaymentRequest.jsx
- |— PurchaseOrder.jsx
- |— VendorPayments.jsx
- |— VendorRestrictions.jsx
- |— admin/
 - |— AdminPoApproval.jsx
 - |— AdminVendorRestrictionApproval.jsx
 - |— DiscountedEstimateApproval.jsx
 - |— EstimateApprovalHistoryTable.jsx
 - |— VendorPaymentApproval.jsx
- |— assets/
 - |— CORPSEED.webp
 - |— index.html
 - |— pan-card.png
 - |— react.svg
 - |— save.png
 - |— signature.png
 - |— template.png
- |— components/
 - |— ui/
 - |— card.jsx
 - |— chart.jsx
 - |— AddressFields.jsx
 - |— BackButton.jsx
 - |— BaseAmountCalculator.jsx
 - |— BulkFileUploader.jsx
 - |— CustomSearchInput.jsx
 - |— CustomSelect.jsx
 - |— DebitNoteView.jsx
 - |— DotIcon.jsx
 - |— EstimatePreview.jsx
 - |— EstimateView.jsx
 - |— FileUploader.jsx
 - |— FormInput.jsx
 - |— FormSelect.jsx
 - |— icons.jsx
 - |— ImageGroup.jsx
 - |— InvoiceView.jsx
 - |— LoadingSpinner.jsx
 - |— navbar.jsx
 - |— NewSelect.jsx
 - |— NewTextEditor.jsx
 - |— NotificationBell.jsx
 - |— NotificationSocketProvider.jsx
 - |— PreviewComponent.jsx
 - |— primitives.ts
 - |— PurchaseInvoiceView.jsx
 - |— RoundedTabs.jsx
 - |— Section.jsx
 - |— SingleFileUploader.jsx
 - |— StatusDisplay.jsx
 - |— TaxInvoice.jsx

```
| theme-switch.jsx
|   └─ UnbilledView.jsx
├─ config/
|   └─ site.ts
├─ dashboards/
|   └─ accounts/
|       └─ AccountsDashboard.jsx
|   └─ charts/
|       └─ ConversionStatus.jsx
|       └─ ConvertedLeadsDataChart.jsx
|       └─ LeadDataChart.jsx
|       └─ LeadStatusChart.jsx
|       └─ ProjectsDataChart.jsx
|       └─ RevenueChart.jsx
|       └─ TopSellLeads.jsx
|       └─ UserLeadStatus.jsx
|   └─ operations/
|       └─ OperationsDashboard.jsx
|   └─ sales/
|       └─ RevenueByService.jsx
|       └─ SalesDashboard.jsx
|       └─ TopCompanies.jsx
|       └─ TopConvertedLeads.jsx
|       └─ TopSellingService.jsx
|   └─ AdminDashboards.jsx
|   └─ LeadDashboardDetail.jsx
|   └─ QualityDashboard.jsx
|   └─ SalesDashboard.jsx
├─ home/
|   └─ HomePage.jsx
├─ hr/
|   └─ Rating.jsx
|   └─ Services.jsx
|   └─ UserApprovals.jsx
|   └─ UsersList.jsx
├─ industry/
|   └─ BusinessActivity.jsx
|   └─ Categories.jsx
|   └─ Industries.jsx
|   └─ SubIndustries.jsx
├─ layouts/
|   └─ default.jsx
|   └─ Layoutpage.jsx
|   └─ NavItems.jsx
|   └─ Sidebar.jsx
├─ legal/
|   └─ CompanyDocuments.jsx
|   └─ OperationsLegal.jsx
|   └─ PaymentVerification.jsx
|   └─ ProcurementVendors.jsx
├─ lib/
|   └─ utils.js
├─ login/
|   └─ ForgotPassword.jsx
|   └─ Login.jsx
|   └─ Otp.jsx
├─ notification/
|   └─ Notifications.jsx
├─ operation/
|   └─ approval/
```

- └─ ManagerApprovals.jsx
- └─ certification/
 - └─ CertificationModule.jsx
- └─ components/
 - └─ commonFunctions.js
- └─ dashboard/
 - └─ Dashboards.jsx
- └─ expenses/
 - └─ Expenses.jsx
- └─ legal/
 - └─ LegalRequests.jsx
- └─ projects/
 - └─ CreatePurchaseOrderModal.jsx
 - └─ ForceCloserAndReopen.jsx
 - └─ ProjectActivities.jsx
 - └─ ProjectDetails.jsx
 - └─ ProjectPR.jsx
 - └─ ProjectPurchaseOrder.jsx
 - └─ Projects.jsx
 - └─ PurchaseOrderView.jsx
- └─ Settings/
 - └─ Departments.jsx
 - └─ Documents.jsx
 - └─ Milestone.jsx
 - └─ OperationsSettings.jsx
 - └─ UserMapWithProduct.jsx
- └─ procurement/
 - └─ solutions/
 - └─ ProcurementSolutions.jsx
 - └─ ProcuremntSolutionDetailPage.jsx
 - └─ Quote.jsx
 - └─ RequestForQuotation.jsx
 - └─ RFQVendors.jsx
 - └─ SolutionOverview.jsx
 - └─ VendorsData.jsx
- └─ quality/
 - └─ IVR.jsx
 - └─ IVRReport.jsx
 - └─ LeadSearch.jsx
- └─ routings/
 - └─ AccountsModuleRouting.jsx
 - └─ DashboardRouting.jsx
 - └─ ERPSettingRouting.jsx
 - └─ HRModuleRouting.jsx
 - └─ IndustryModuleRouting.jsx
 - └─ OperationModuleRouting.jsx
 - └─ ProcurementRouting.jsx
 - └─ QualityRouting.jsx
 - └─ SalesModuleRouting.jsx
- └─ sales/
 - └─ company/
 - └─ BasicCompany.jsx
 - └─ Company.jsx
 - └─ CompanyAndUnitsInLead.jsx
 - └─ CompanyGstList.jsx
 - └─ CompanyLeads.jsx
 - └─ CompanyProjects.jsx
 - └─ CompanyUnitDetails.jsx
 - └─ CompanyUnits.jsx
 - └─ CreateCompanyForm.jsx

- └─ CreateLeadCompanyForm.jsx
 - └─ FullCompanyDetailsForm.jsx
 - └─ UnitDetails.jsx
- └─ estimate/
 - └─ Estimate.jsx
 - └─ EstimatePaymentRegister.jsx
- └─ invoice/
 - └─ SalesAdvanceInvoice.jsx
- └─ leads/
 - └─ leadEstimate/
 - └─ EstimateFormSchema.jsx
 - └─ LeadEstimates.jsx
 - └─ NewEstimatePreview.jsx
 - └─ ProductFormFieldsDetails.jsx
 - └─ ServiceFormFieldsDetail.jsx
 - └─ AllTasks.jsx
 - └─ AutoHistory.jsx
 - └─ AutomationStatus.jsx
 - └─ ChildLead.jsx
 - └─ CreateLead.jsx
 - └─ DiscountedEstimate.jsx
 - └─ LeadDetail.jsx
 - └─ LeadEstimate.jsx
 - └─ LeadHistory.jsx
 - └─ LeadInfo.jsx
 - └─ Leads.jsx
 - └─ LeadTask.jsx
 - └─ Projects.jsx
 - └─ SalesReport.jsx
 - └─ ServicePaymentTerm.jsx
 - └─ ServingCompanies.jsx
- └─ payment/
 - └─ SalesPOPayment.jsx
- └─ projects/
 - └─ SalesProject.jsx
- └─ proposal/
 - └─ AllProposal.jsx
 - └─ Proposal.jsx
- └─ unbill/
 - └─ SalesUnbill.jsx
- └─ vendors/
 - └─ Vendors.jsx
- └─ setting/
 - └─ applicantType/
 - └─ ApplicantTypes.jsx
 - └─ comments/
 - └─ LeadComments.jsx
 - └─ contacts/
 - └─ ContactModalForm.jsx
 - └─ country/
 - └─ CityData.jsx
 - └─ CountryData.jsx
 - └─ StateData.jsx
 - └─ department/
 - └─ Department.jsx
 - └─ designation/
 - └─ Designation.jsx
 - └─ ipaddress/
 - └─ IpAddress.jsx
 - └─ paymentTerm/

```
└─ PaymentTems.jsx
└─ procurement/
  └─ ProcurementCategory.jsx
  └─ ProcurementSubCategory.jsx
└─ products/
  └─ BusinessArrangement.jsx
  └─ ProductAttachedDocument.jsx
  └─ ProductCategory.jsx
  └─ ProductDocument.jsx
  └─ ProductMilestones.jsx
  └─ ProductServiceDetails.jsx
  └─ ProductSubCategory.jsx
  └─ ProductTAT.jsx
  └─ SalesTAT.jsx
  └─ SolutionDetails.jsx
  └─ SolutionPrice.jsx
  └─ Solutions.jsx
└─ proposalAndBrouchers/
  └─ category/
    └─ ProposalCategory.jsx
  └─ menu/
    └─ ProposalMenu.jsx
  └─ service/
    └─ ProposalService.jsx
  └─ subCategory/
    └─ ProposalSubCategory.jsx
└─ proposalAndTemplates/
  └─ TemplatesAndEmailBody.jsx
└─ slug/
  └─ Slug.jsx
└─ status/
  └─ LeadStatus.jsx
└─ urls/
  └─ Urls.jsx
└─ styles/
  └─ globals.css
└─ toolkit/
  └─ slices/
    └─ accountSlice.js
    └─ authSlice.js
    └─ commonSlice.js
    └─ companySlice.js
    └─ dashboardSlice.js
    └─ leadSlice.js
    └─ operationSlice.js
    └─ organizationSlice.js
    └─ productSlice.js
    └─ settingSlice.js
    └─ vendorsSlice.js
  └─ store.js
└─ types/
  └─ index.ts
└─ users/
  └─ DeactiveUserList.jsx
  └─ UserHistory.jsx
  └─ UserMailConfig.jsx
  └─ UserManagerApproval.jsx
  └─ Users.jsx
└─ vendor-request/
  └─ VendorEstimate.jsx
```

```
| |— VendorPaymentHistory.jsx
| |— VendorRequestDetail.jsx
| |— VendorRequests.jsx
| |— Vendors.jsx
| |— VendorsPaymentForProcurement.jsx
|— App.jsx
|— common.js
|— data.js
|— heroui-shim.js
|— httpRequest.js
|— main.jsx
|— ProtectedRoute.jsx
|— uipvider.jsx
|— useNotificationSocket.js
|— vite-env.d.ts
```

8. Module-by-Module Reference

This section documents the purpose of every folder under `src/` and the responsibility of its key files.

8.1 Sales — `src/sales/`

The CRM heart of the ERP: lead capture, qualification, estimation, proposals, projects and billing on the sales side.

File	Purpose
<code>leads/Leads.jsx</code>	Main lead list/board (kept alive across navigation).
<code>leads/LeadDetail.jsx</code>	Layout route hosting all per-lead tabs.
<code>leads/LeadInfo.jsx</code>	Core lead information form.
<code>leads/CreateLead.jsx</code>	New lead creation form.
<code>leads/ChildLead.jsx</code>	Linked/child lead management.
<code>leads/AllTasks.jsx</code>	Cross-lead task board.
<code>leads/LeadTask.jsx</code>	Tasks for a single lead.
<code>leads/LeadHistory.jsx</code> / <code>AutoHistory.jsx</code>	Activity timeline / automation activity log.
<code>leads/AutomationStatus.jsx</code>	Automation engine on/off + status report.
<code>leads/DiscountedEstimate.jsx</code>	Lists estimates issued with a discount.
<code>leads/LeadEstimate.jsx</code> + <code>leadEstimate/*</code>	Estimate builder for a lead (schema, product/service line items, preview).
<code>leads/ServicePaymentTerm.jsx</code>	Per-service payment-term configuration.
<code>leads/ServingCompanies.jsx</code>	Companies currently being serviced.
<code>leads/SalesReport.jsx</code>	Sales performance report.
<code>leads/Projects.jsx</code>	Sales-side project listing (referenced by routing).
<code>company/Company.jsx</code> + <code>CompanyLeads/CompanyProjects/CompanyUnits*</code>	Company master, its GST units, and the leads/projects under each unit.
<code>company/CreateCompanyForm.jsx</code> / <code>CreateLeadCompanyForm.jsx</code> / <code>FullCompanyDetailsForm.jsx</code>	Company creation/attachment forms (new company vs. existing).

estimate/Estimate.jsx	All-estimates list.
estimate/EstimatePaymentRegister.jsx	Payment register tied to estimates.
proposal/Proposal.jsx / AllProposal.jsx	Single-lead proposal editor / all-proposals list.
invoice/SalesAdvanceInvoice.jsx	Advance tax invoice screen (sales side).
payment/SalesPOPayment.jsx	PO payment tracking (sales side).
projects/SalesProject.jsx	Sales-side project tracking.
unbill/SalesUnbill.jsx	Work completed but not yet invoiced.
vendors/Vendors.jsx	Vendors tab inside a lead.

8.2 Accounts — src/accounts/

Finance & books of account — approvals, invoicing, payments, taxation and the full double-entry ledger system.

File	Purpose
CompanyApprovals.jsx / CompanyForm.jsx / CompanyUnitsInAccount.jsx	Accounts-team review & approval of new companies/units.
Expense.jsx	Expense approvals.
PaymentApprovals.jsx	Generic payment approval queue.
PurchaseOrder.jsx	PO payments list.
ProcurementPaymentRequest.jsx	Procurement payment requests awaiting accounts action.
VendorPayments.jsx / VendorRestrictions.jsx	Vendor payment tracking and vendor payment-hold restrictions.
AdvanceInvoices.jsx / AdvanceTaxInvoiceView.jsx	Advance tax invoice list & printable view.
CreditNote.jsx	Credit note issuance.
vendorDetails/VendorDetails.jsx	Master vendor records used across accounts.
ledgers/Ledger.jsx, LedgerEntriesPage.jsx, LedgerMasterSection.jsx	Ledger master list, entry drill-down, and master-data section.
organization/Organizations.jsx	Layout route hosting the entire double-entry books sub-app.

organization/Group.jsx / GroupLedger.jsx / Ledger.jsx / LedgerDetail.jsx	Chart-of-accounts groups and ledgers.
organization/Voucher.jsx / DailyBook.jsx	Voucher entry and the day book.
organization/ProfitLoss.jsx / CashFlow.jsx / BalanceSheet.jsx / TrailBalance.jsx	Standard financial statements.
organization/GST.jsx / TDS.jsx / Taxation.jsx	Tax reporting.
organization/AllInvoice.jsx / PurchaseInvoices.jsx / Unbill.jsx / InvoicesByUnbilled.jsx	Sales & purchase invoice registers, unbilled work.
organization/DebitNotes.jsx / BankStatement.jsx / PaymentRegister.jsx / SalesReport.jsx / OrganizationEstimate.jsx / OrganizationDetail.jsx	Debit notes, bank reconciliation, payment register, sales report, org profile.
organization/settings/LedgerType.jsx / VoucherType.jsx / Statutory.jsx	Master configuration for ledger types, voucher types and statutory compliance types.

8.3 Procurement — src/procurement/ (+ src/vendor-request/)

Vendor sourcing and quotation management. Works together with src/vendor-request/, which holds the earlier-stage vendor request/estimate/payment screens shared with Accounts.

File	Purpose
solutions/ProcurementSolutions.jsx	List of solutions requiring vendor quotations.
solutions/ProcuremntSolutionDetailPage.jsx	Layout route for a solution's procurement workflow.
solutions/SolutionOverview.jsx	Overview tab of a procurement solution.
solutions/VendorsData.jsx	Candidate vendor data for the solution.
solutions/RequestForQuotation.jsx	Create/manage an RFQ.
solutions/RFQVendors.jsx	Vendors invited to a given RFQ.
solutions/Quote.jsx	A single vendor's submitted quotation.
vendor-request/VendorRequests.jsx / VendorRequestDetail.jsx	Incoming vendor requests list & detail (linked to a lead).
vendor-request/Vendors.jsx / VendorsPaymentForProcurement.jsx	Procurement's vendor master list & payment tracking.
vendor-request/VendorEstimate.jsx / VendorPaymentHistory.jsx	Vendor estimates and payment history.

8.4 Operation — src/operation/

Project execution once a deal is won: project lifecycle, milestones, legal checks, manager approvals and department settings.

File	Purpose
projects/Projects.jsx / ProjectDetails.jsx / ProjectActivities.jsx	Project list, detail workspace and activity log.
projects/ProjectPurchaseOrder.jsx / CreatePurchaseOrderModal.jsx / PurchaseOrderView.jsx	Raising and viewing a project's purchase order.
projects/ProjectPR.jsx	Procurement payment request tied to a project's PO.
projects/ForceCloserAndReopen.jsx	Force-close / reopen a project (needs approval).
approval/ManagerApprovals.jsx	Manager approval queue for operations actions.
legal/LegalRequests.jsx	Legal requests raised from operations.
certification/CertificationModule.jsx	Certification tracking for a project/service.
expenses/Expenses.jsx	Operations expense submission/approval.
dashboard/Dashboards.jsx	Operation-specific dashboard widgets.
Settings/OperationsSettings.jsx	Layout route for operation settings tabs.
Settings/UserMapWithProduct.jsx	Maps users → products/solutions they can operate on.
Settings/Milestone.jsx / Documents.jsx / Departments.jsx	Milestone templates, document requirements, department master data.
components/commonFunctions.js	Shared helper functions for the operation module.

8.5 Quality — src/quality/

Lead quality assurance: call verification (IVR) and lead search/audit, used by the Quality team and re-used by Sales/Admin dashboards.

File	Purpose
IVR.jsx	Interactive voice response call verification screen.
IVRReport.jsx	IVR reporting/analytics.
LeadSearch.jsx	Cross-lead search used for QA audits.

8.6 HR — src/hr/

Internal HR administration: employee list, approvals, service catalogue and ratings.

File	Purpose
UsersList.jsx	HR view of all employees.
UserApprovals.jsx	New-employee / change approvals.
Services.jsx	Internal HR service catalogue.
Rating.jsx	Employee/service rating screen.

8.7 Industry — src/industry/

Master data describing the industries and business activities a client company can belong to; feeds dropdowns across Sales/Accounts company forms.

File	Purpose
Industries.jsx	Top-level industry list.
SubIndustries.jsx	Sub-industry list (child of an industry).
Categories.jsx	Category master under an industry.
BusinessActivity.jsx	Business-activity master data.

8.8 Legal — src/legal/

Compliance review layer, restricted to ADMIN and “Legal Executive” users.

File	Purpose
OperationsLegal.jsx	Legal review of requests raised by Operations.
ProcurementVendors.jsx	Legal review of vendors used by Procurement.
CompanyDocuments.jsx	Legal document repository per company.
PaymentVerification.jsx	Legal sign-off on payments before release.

8.9 Admin — src/admin/

Cross-cutting, high-authority approval screens available to ADMIN users regardless of department.

File	Purpose
AdminPoApproval.jsx	Final approval of purchase orders.

VendorPaymentApproval.jsx	Final approval of vendor payments.
DiscountedEstimateApproval.jsx	Approval for estimates issued with a discount.
AdminVendorRestrictionApproval.jsx	Approval to place/lift a hold on a vendor.
EstimateApprovalHistoryTable.jsx	Shared history table used by the approval screens above.

8.10 Users — src/users/

ERP account administration (distinct from the HR module — this manages login accounts, not HR records).

File	Purpose
Users.jsx	All ERP users list (create/edit accounts, assign roles/department).
UserManagerApproval.jsx	Manager approval queue for user account changes.
DeactiveUserList.jsx	Deactivated accounts.
UserMailConfig.jsx	Per-user email/SMTP configuration.
UserHistory.jsx	Audit history for a single user account.

8.11 Setting — src/setting/

The master-data / configuration backbone consumed by every other module — statuses, product & solution catalogue, geography, proposal templates, and more.

File	Purpose
status/LeadStatus.jsx	Lead status master (pipeline stages).
comments/LeadComments.jsx	Predefined comment templates for leads.
contacts/ContactModalForm.jsx	Reusable contact-person modal form.
country/CountryData.jsx, StateData.jsx, CityData.jsx	Country → State → City geography master data.
department/Department.jsx / designation/Designation.jsx	Department and designation master data.
ipaddress/IpAddress.jsx	Allow-listed IP addresses.
paymentTerm/PaymentTems.jsx	Payment-term master data.
procurement/ProcurementCategory.jsx, ProcurementSubCategory.jsx	Procurement category taxonomy.

products/Solutions.jsx, SolutionDetails.jsx, SolutionPrice.jsx	Solution catalogue — the sellable service packages and their pricing.
products/BusinessArrangement.jsx, ProductCategory.jsx, ProductSubCategory.jsx	Product taxonomy under a solution's business arrangement.
products/ProductDocument.jsx, ProductAttachedDocument.jsx, ProductMilestones.jsx, ProductServiceDetails.jsx, ProductTAT.jsx, SalesTAT.jsx	Per-product document requirements, milestones, service details and turn-around-time targets.
proposalAndBrouchers/menu, category, subCategory, service	Menu → category → sub-category → service taxonomy used to build the Proposal menu.
proposalAndTemplates/TemplatesAndEmailBody.jsx	Email/proposal template authoring (rich text).
applicantType/ApplicantTypes.jsx	Applicant-type master data.
slug/Slug.jsx / urls/Urls.jsx	URL slug and marketing-URL master data.

8.12 Dashboards — src/dashboards/

Aggregated analytics views. Each department gets its own entry route (see §6.1) but most compose the same chart components.

File	Purpose
AdminDashboards.jsx	Org-wide admin dashboard (also reused for the Quality landing).
LeadDashboardDetail.jsx	Drill-down into a month's lead data from any dashboard.
QualityDashboard.jsx / SalesDashboard.jsx	Top-level module dashboards.
accounts/AccountsDashboard.jsx	Accounts dashboard.
operations/OperationsDashboard.jsx	Operations dashboard.
sales/SalesDashboard.jsx, RevenueByService.jsx, TopCompanies.jsx, TopConvertedLeads.jsx, TopSellingService.jsx	Sales dashboard and its widget components.
charts/ConversionStatus.jsx, ConvertedLeadsDataChart.jsx, LeadDataChart.jsx, LeadStatusChart.jsx, ProjectsDataChart.jsx, RevenueChart.jsx, TopSellLeads.jsx, UserLeadStatus.jsx	Shared Recharts-based chart building blocks reused across all dashboards.

8.13 Home, Login & Notification — src/home/, login/, notification/

Entry surfaces outside the protected ERP shell, plus the notification centre.

File	Purpose
home/HomePage.jsx	Public landing page at "/".
login/Login.jsx	Login form (see §4).
login/ForgotPassword.jsx / Otp.jsx	Password-reset request and OTP verification.
notification/Notifications.jsx	Full notification list/centre.

8.14 Shared Layout, Routing & Components — src/layouts/, routings/, components/

Cross-cutting infrastructure used by every module above.

File	Purpose
layouts/Layoutpage.jsx	The authenticated app shell — sidebar + header + breadcrumb + Outlet (see §5).
layouts/Sidebar.jsx	Renders the department-specific nav item tree.
layouts/NavItems.jsx	All nav-item arrays (navItems, salesNavItems, accountNavItems, ...).
layouts/default.jsx	Fallback/default layout wrapper.
routings/*.jsx	One file per module, each exporting a function that returns its <Route> fragment (see §6).
components/FormInput.jsx, FormSelect.jsx, CustomSelect.jsx, NewSelect.jsx, CustomSearchInput.jsx, AddressFields.jsx	Reusable form controls.
components/FileUploader.jsx, SingleFileUploader.jsx, BulkFileUploader.jsx, ImageGroup.jsx	File/image upload widgets.
components/EstimateView.jsx, EstimatePreview.jsx, InvoiceView.jsx, PurchaseInvoiceView.jsx, TaxInvoice.jsx, DebitNoteView.jsx, UnbilledView.jsx, PreviewComponent.jsx	Printable / shareable document preview components used across Sales & Accounts.
components/NewTextEditor.jsx	TipTap-based rich text editor used in proposals/templates.
components/NotificationBell.jsx, NotificationSocketProvider.jsx	Realtime notification UI (see §3.3).

components/BaseAmountCalculator.jsx, BulkFileUploader.jsx, StatusDisplay.jsx, RoundedTabs.jsx, Section.jsx, DotIcon.jsx, BackButton.jsx, LoadingSpinner.jsx, theme-switch.jsx, navbar.jsx, icons.jsx	General-purpose UI building blocks.
components/ui/card.jsx, chart.jsx	Low-level presentational primitives (shadcn-style) used by the dashboard charts.

8.15 State Management — Redux Toolkit — src/toolkit/

Global client state, one slice per business domain. See §9 for details.

File	Purpose
store.js	Store configuration, redux-persist wiring, logout state-wipe.
slices/authSlice.js	Authentication, session, department lookup, automation on/off.
slices/leadSlice.js	Leads, lead detail, lead tasks/history.
slices/companySlice.js	Companies, GST units.
slices/accountSlice.js	Accounts-module data (approvals, invoices, payments).
slices/organizationSlice.js	The full ledger/books-of-account domain (largest slice).
slices/vendorsSlice.js	Vendors, vendor requests, RFQs, quotations, vendor payments.
slices/operationSlice.js	Projects, PO/PR, operation settings.
slices/productSlice.js	Solutions/products catalogue.
slices/settingSlice.js	All Settings-module master data.
slices/dashboardSlice.js	Dashboard aggregation data.
slices/commonSlice.js	Cross-cutting shared state (e.g. rating updates).

8.16 Root-level infrastructure files — src/(src/ root)

Files living directly under src/ that don't belong to a single module.

File	Purpose
main.jsx	React root render, Redux Provider/PersistGate, UIProvider, BrowserRouter.
App.jsx	Top-level route tree (see §3.1 / §6).
ProtectedRoute.jsx	Route guard for the whole /erp/:userId subtree (see §4).
httpRequest.js	Shared Axios instance (see §3.2).

useNotificationSocket.js	STOMP/WebSocket notification hook.
uiprovider.jsx	HeroUI theme/provider wrapper.
heroui-shim.js	Global size/font override shim so HeroUI components render consistently app-wide without per-file edits.
common.js / data.js	Shared constants / static reference data.
lib/utils.js	Small generic utility functions (e.g. class-name merging).
config/site.ts	Site-level configuration constants.
types/index.ts / vite-env.d.ts	Shared TypeScript type declarations.
styles/globals.css	Tailwind CSS entry point and global styles.

9. State Management (Redux Toolkit)

`src/toolkit/store.js` combines eleven slice reducers. All slices except `auth` are persisted to `localStorage` under the `"persist:root"` key via `redux-persist`; the `auth` slice is persisted separately to `sessionStorage` so that closing the browser naturally logs the user out.

Slice	State domain
<code>auth</code>	Login state, JWT, roles, current user, department lookup, automation on/off.
<code>leads</code>	Lead list/detail data and related async thunks.
<code>common</code>	Cross-cutting shared UI/data state.
<code>setting</code>	All Settings-module master data (status, solutions, geography, templates, ...).
<code>company</code>	Companies and their GST units.
<code>account</code>	Accounts-module approvals, invoices, payments.
<code>vendors</code>	Vendors, vendor requests, RFQs/quotations, vendor payments.
<code>dashboard</code>	Dashboard aggregation data for charts.
<code>organization</code>	Full ledger / books-of-account domain (groups, ledgers, vouchers, statements).
<code>product</code>	Solutions/products catalogue.
<code>operation</code>	Projects, PO/PR, operation settings.

10. End-to-End Business Flow (“Lead to Ledger”)

This is the core operating flow the ERP is built around. Each step names the business action, the module that owns it, and the primary screen(s) involved.

1. Lead capture (Sales) — A sales rep creates a lead (sales/leads → Leads.jsx, CreateLead.jsx). The lead carries basic contact info, an industry/sub-industry/category (Industry module master data) and starts in the first Lead Status stage (Settings → Status).
2. Quality verification (Quality) — In parallel, the Quality team can pull the same lead via quality/leads or quality/leadsSearch, run an IVR call-verification (quality/ivr), and log results (quality/report). LeadSearch supports cross-team audit of any lead.
3. Estimation (Sales) — The rep builds an estimate against the lead (leadEstimate / leadEstimates), picking Solutions and their line-item Products/Services and prices from the Settings → Solutions catalogue (settings/solutions, businessArrangement, productCategory). A shareable, unauthenticated preview link is generated (/:leadId/:uuid/estimate-preview → EstimatePreview.jsx) for the client to review.
4. Discount approval (Admin) — If the estimate carries a discount, it appears in sales/discountedEstimate and must be cleared by an ADMIN at admin/discountedEstimate → DiscountedEstimateApproval.jsx before it can proceed.
5. Proposal (Sales) — Once the estimate is accepted, a formal Proposal is drafted (sales/proposal, Proposal.jsx / AllProposal.jsx) using templates authored in Settings → Proposal & Templates.
6. Lead → Company conversion (Sales → Accounts) — A won lead is converted into a Company record (CreateCompanyForm.jsx / CreateLeadCompanyForm.jsx), including its GST registration units (CompanyGstList, CompanyUnits, CompanyUnitDetails). The new company/unit is then reviewed and approved by Accounts (accounts/companyApprovals → CompanyApprovals.jsx, CompanyUnitsInAccount.jsx).
7. Project creation (Operation) — The won, approved deal becomes a Project (operation/projects → Projects.jsx → ProjectDetails.jsx), tracked against the milestone template configured for its product/solution (operation/settings/milestones). Sales keeps a parallel view (sales/projects) and Procurement can see the same project when vendor work is required (procurement/projects).
8. Vendor sourcing (Procurement) — If the project needs outside vendor delivery, Procurement raises a Vendor Request (procurement/vendors-requests → VendorRequests.jsx / VendorRequestDetail.jsx), identifies candidate vendors (procurement/vendorList → Vendors.jsx / VendorsData.jsx), issues a Request For Quotation (RequestForQuotation.jsx), invites vendors (RFQVendors.jsx) and records each vendor's Quote (Quote.jsx) under procurement/vendor-quotations.

9. Purchase order & payment request (Operation → Accounts → Admin) — Operation raises a Purchase Order against the winning vendor quote (ProjectPurchaseOrder.jsx / CreatePurchaseOrderModal.jsx) and a linked Procurement Payment Request (ProjectPR.jsx). These land in Accounts as accounts/procurementPaymentRequests and accounts/poPayments, and require final sign-off by an ADMIN at admin/poApproval → AdminPoApproval.jsx.
10. Vendor payment (Accounts → Admin) — Once the PO is approved, Accounts tracks the vendor payment (accounts/vendorsPayment → VendorPayments.jsx, with history at VendorPaymentHistory.jsx). Payment release needs ADMIN approval (admin/vendorPaymentApproval) and can be blocked entirely if the vendor is on hold (accounts/vendorRestrictions, admin/vendorRestrictions).
11. Legal review (Legal, parallel track) — Any legal question raised during operations (operation/legalRequests → LegalRequests.jsx) is reviewed by the Legal module (legal/operationsRequests), along with vendor legal status (legal/vendors), company document compliance (legal/companyDocuments) and pre-payment legal verification (legal/paymentVerification) — all restricted to ADMIN / Legal Executive.
12. Delivery & manager approvals (Operation) — As work completes, Operation records activities (ProjectActivities.jsx), and any exception — expense claims (operation/expenses), force-closing or reopening a project (operation/reopenCloseApprovals) — goes through the manager approval queue (operation/approvals → ManagerApprovals.jsx).
13. Billing (Sales → Accounts) — Completed-but-not-yet-invoiced work sits in Unbilled (sales/unbilled / accounts/unbilled → Unbill.jsx); converting it produces entries in All Invoice, Purchase Invoices, Advance Tax Invoices, Credit Notes and Debit Notes.
14. Books of account (Accounts → Organizations) — Every financial event ultimately posts into the double-entry books: Groups → Ledgers → Ledger Entries → Vouchers → Day Book, rolling up into Trial Balance, Profit & Loss, Cash Flow, Balance Sheet, GST and TDS reports, the Sales Report, Payment Register and Bank Statement (all under accounts/organizations/*).
15. Oversight (Dashboards, all along) — At every stage above, the relevant dashboard (Admin/Sales/Accounts/Operations/Quality) rolls the same data up into charts — conversion status, revenue by service, top leads/companies/services — so management can see the pipeline end-to-end without leaving the dashboard module.

Note: Running alongside the pipeline above (not part of the sequential flow, but touching every step): HR manages employee onboarding/approvals/ratings; the Users module manages ERP login accounts, approvals and deactivation; and Settings supplies almost every dropdown and master-data value used by every screen described above.

11. Build, Lint & Tooling

npm script	Purpose
dev	Starts the Vite dev server.
build	Runs `tsc` (type-check) then `vite build` for production.
lint	Runs ESLint with --fix.
preview	Serves the production build locally.

Configuration lives at the repository root: vite.config.ts, tailwind.config.js, postcss.config.js, eslint.config.mjs, tsconfig.json / tsconfig.node.json, components.json (shadcn/HeroUI component registry) and vercel.json (deployment target).